

RTVIO: Single-Pass Georeferenced 3D Reconstruction from UAV Video Using a Feed-Forward Geometry Transformer

Prof. Priyanka Jain, Mr. Kaustubh Ranjan Singh

Joel Sam, Gokul G, Kashif Ali Khan, Harsh Bhatia, Riddhima Gupta, Bhavya Sahni

Department of Computer Engineering
Delhi Technological University, New Delhi, India

September 2026

Abstract

Producing an accurate, georeferenced three-dimensional model from a single unmanned-aerial-vehicle pass is conventionally a slow, multi-stage photogrammetric process built on Structure-from-Motion and Multi-View Stereo. This paper presents RTVIO, an end-to-end capture and reconstruction system that replaces that stack with a single feed-forward geometry transformer. Video is captured either from a MAVLink drone over RTSP or from an Android handset that also streams inertial and satellite-navigation data, and is reconstructed on a consumer GPU into a dense coloured point cloud, a true Screened-Poisson surface mesh, and a georeferenced elevation raster. The central technical contribution is the treatment of window-to-window consistency: because the network predicts a depth for every pixel of the frames shared between two overlapping inference windows, hundreds of thousands of exact correspondences are available without any feature matching, and a robust similarity transform fitted to them corrects scale at every seam. We report measurements from seven complete reconstructions on an NVIDIA RTX 5070 Ti, spanning 201 to 6,891 frames, at 4.4–7.0 frames per second and up to 16.9 million points. We also report, candidly, two unresolved gaps: a Monte-Carlo analysis showing that consumer-grade satellite-navigation noise alone keeps georeferencing error above the one-metre target, and a throughput shortfall against a fifteen-minute processing budget.

1. OBJECTIVE

The objective of this work is to design, implement, and evaluate a complete system that converts a single-pass unmanned-aerial-vehicle (UAV) video, together with whatever satellite-navigation data accompanies it, into an accurate georeferenced three-dimensional model, within a processing budget short enough to be operationally useful.

The work is built against Smart India Hackathon 2026 problem statement PS-17, *Single-Pass Drone Video to Accurate 3D Model Generation* [17], which specifies single-pass drone video and GPS as mandatory inputs, inertial data, camera intrinsics and RTK corrections as optional inputs, and requires a georeferenced mesh or point cloud with a spatial accuracy of one metre or better, produced in under fifteen minutes for a ten-minute video. The problem statement weights its evaluation as accuracy 30%, completeness 20%, speed 20%, innovation 15%, scalability 10%, and user interface 5%.

1.1. A Capture-Agnostic Acquisition Front End

To build an acquisition layer that treats a consumer smartphone and a MAVLink-equipped drone as interchangeable sources, writing both into a single on-disk session format so that one reconstruction path serves both, and so that the absence of specialised flight hardware is not a barrier to producing a model.

1.2. Feed-Forward Geometric Reconstruction

To replace the conventional Structure-from-Motion and Multi-View Stereo pipeline with a single feed-forward transformer that jointly predicts camera intrinsics, extrinsics and dense per-pixel depth, thereby eliminating incremental bundle adjustment and the failure modes associated with it.

1.3. Scale-Consistent Fusion Across Inference Windows

To develop a method for composing the independent, arbitrarily-scaled predictions of successive inference windows into one globally consistent model, and to quantify the residual scale error that remains at each seam.

1.4. Georeferencing and Honest Error Accounting

To anchor the reconstruction to the Earth through a similarity transform fitted to satellite-navigation fixes, and to characterise quantitatively how much of the one-metre accuracy target is consumed by satellite-navigation noise alone, before any other error source is considered.

2. RELATED WORK

2.1. Classical Structure-from-Motion

The established approach to reconstruction from unordered or sequential imagery is incremental Structure-from-Motion

From one flight to a georeferenced 3D model

Video, IMU and GPS leave a phone over Wi-Fi. A GPU desktop turns them into a dense point cloud, a textured mesh and a UTM elevation raster, then shows the result in a browser.

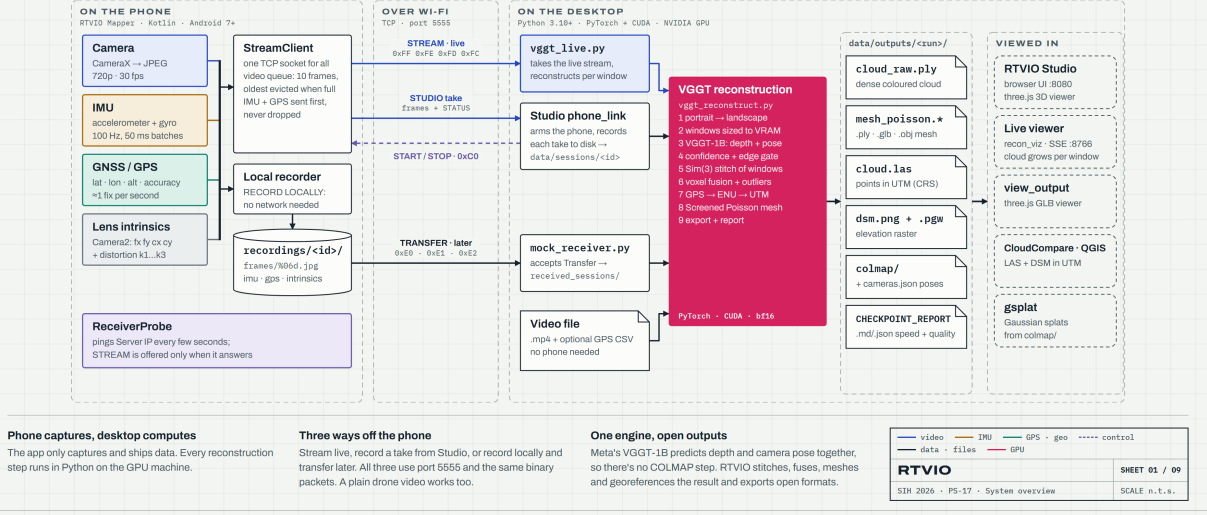


Figure 1: System overview: from a single flight to a georeferenced 3D model. Capture may originate from an Android handset or directly from a MAVLink drone; all reconstruction occurs on the desktop GPU.

followed by Multi-View Stereo. COLMAP [4] is the reference implementation and remains the accuracy benchmark. Its cost model, however, is poorly suited to a time-bounded task: incremental reconstruction repeatedly re-triangulates and re-optimises as images are registered, and its runtime scales super-linearly with image count. Feature matching also degrades sharply on the low-texture, motion-blurred, repetitive imagery characteristic of aerial survey.

2.2. Visual-Inertial Odometry

Where a pose estimate is needed during flight rather than after it, the standard solutions are filter-based or optimisation-based visual-inertial odometry: the Multi-State Constraint Kalman Filter [11] maintains a sliding window of camera poses in the filter state rather than landmark positions, while VINS-Mono [10] performs tightly coupled non-linear optimisation over inertial pre-integration factors and visual residuals. Both require a well-observed inertial-visual extrinsic calibration and, critically, an absolute position reference to bound drift.

2.3. Feed-Forward 3D Reconstruction

A recent line of work discards the optimisation loop altogether. DUS3R [2] recast pairwise reconstruction as direct regression of point maps, unifying the monocular and binocular cases and removing the requirement for known intrinsics or poses. MAST3R [3] extended the formulation with a dense matching head. VGGT [1] generalised it from image pairs to hundreds of views in a single forward pass, jointly predicting camera parameters, depth maps, point maps and tracks, and it is the model on which this work is built. The

practical consequence is a change of cost regime: reconstruction becomes a bounded, predictable GPU inference cost rather than an open-ended optimisation.

3. SYSTEM ARCHITECTURE

RTVIO comprises three cooperating parts. An Android application written in Kotlin captures camera, inertial and satellite-navigation data and transmits it over a single TCP socket. A Python engine running on a CUDA-capable workstation performs all reconstruction. RTVIO Studio, a browser-based control plane served by the same Python process, drives capture from either source, queues GPU jobs, and renders results.

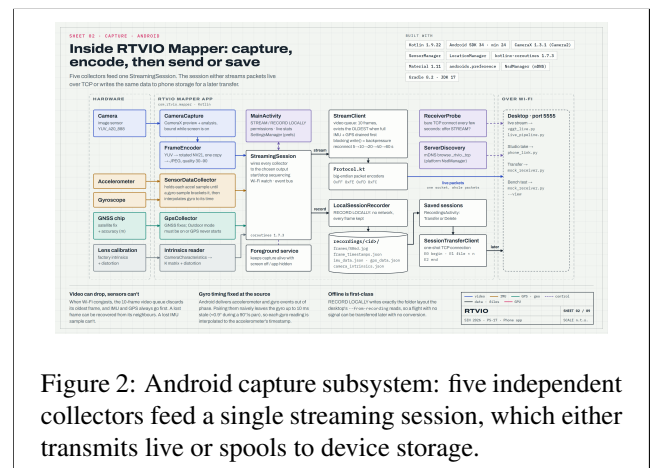


Figure 2: Android capture subsystem: five independent collectors feed a single streaming session, which either transmits live or spools to device storage.

A deliberate architectural decision is that the capture device performs no reconstruction whatsoever. The handset

encodes JPEG frames and serialises sensor samples; the drone supplies an RTSP video stream and MAVLink telemetry. This keeps the mobile power budget low, keeps the wire protocol simple, and concentrates all numerically sensitive work on one machine where it can be tested deterministically.

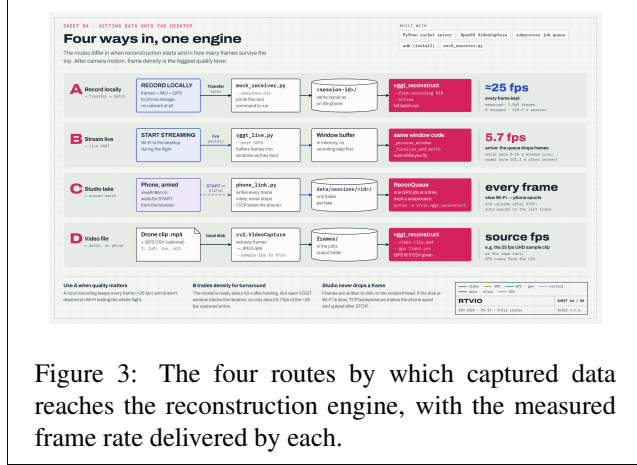


Figure 3: The four routes by which captured data reaches the reconstruction engine, with the measured frame rate delivered by each.

Four acquisition routes exist: remote-controlled recording through Studio, which retains every frame by spooling to device storage when the network falls behind; fully offline recording with later bulk transfer; live streaming into a window-by-window reconstruction; and direct reconstruction of an existing video file. All four converge on one session directory layout, so a single reconstruction implementation serves them all.

4. THEORY

4.1. Camera Model and Lens Calibration

A pinhole camera maps a world point $\mathbf{X}$ to image coordinates through the intrinsic matrix $\mathbf{K}$ and the extrinsic rotation and translation:

$$\mathbf{x} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}] \mathbf{X}, \quad \mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

The inverse operation, unprojecting a pixel (u, v) at depth d into camera coordinates, is used at every pixel of every frame:

$$\mathbf{p}_{\text{cam}} = \left(\frac{u - c_x}{f_x} d, \frac{v - c_y}{f_y} d, d \right) \quad (2)$$

Real lenses depart from this model. For conventional and mildly wide-angle lenses the system uses the Brown–Conrady model, with radial coefficients k_1, k_2, k_3 and tangential coefficients p_1, p_2 applied to the normalised coordinates (x, y) with $r^2 = x^2 + y^2$:

$$x_d = x (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2 (r^2 + 2x^2) \quad (3)$$

$$y_d = y (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y^2) + 2p_2 xy \quad (4)$$

The drone’s camera is a fisheye of approximately 135° to 150° field of view, for which the Brown–Conrady model is inadequate. For this case the Kannala–Brandt generic model [7] is used, which expresses the distorted angle θ_d as an odd polynomial in the incidence angle θ :

$$\theta_d = \theta (1 + k_1 \theta^2 + k_2 \theta^4 + k_3 \theta^6 + k_4 \theta^8) \quad (5)$$

Both models are fitted from checkerboard views using the planar-target method of Zhang [8], and the model achieving the lower root-mean-square reprojection error is retained. The importance of this step is geometric rather than cosmetic: the transformer assumes a pinhole camera, so an uncalibrated fisheye forces it to settle on a compromise focal length. On the first real drone take the network estimated $f_x = 272$ px at 518 px width, about 87° across, and a right-angled wall corner was reconstructed at approximately 150° .

The edge-to-edge field of view implied by a calibrated profile is compared against the naive pinhole estimate

$$\text{FOV}_{\text{pinhole}} = 2 \arctan \left(\frac{w}{2f_x} \right) \quad (6)$$

and the discrepancy is reported in every reconstruction report. When a profile must be rescaled to a different capture resolution, the half-pixel convention is observed, so that the principal point remains consistent under resampling:

$$c'_x = \left(c_x + \frac{1}{2} \right) s_x - \frac{1}{2} \quad (7)$$

4.2. Feed-Forward Geometry Estimation

The reconstruction core is VGGT-1B [1], a transformer whose aggregator has 24 blocks, 16 attention heads and an embedding dimension of 1024, operating on DINOv2 patch embeddings with a patch size of 14 and four register tokens, and alternating frame-wise and global attention. The heads for point maps and tracks are disabled, removing approximately 98 million unused parameters; only the camera and depth heads are evaluated.

Camera parameters are predicted in a nine-dimensional encoding comprising a translation, a quaternion, and two field-of-view angles. The intrinsics are recovered as

$$f_y = \frac{H/2}{\tan(\text{fov}_h/2)}, \quad f_x = \frac{W/2}{\tan(\text{fov}_w/2)} \quad (8)$$

with the principal point assumed to lie at the image centre. This assumption is a known limitation of the encoding and is one reason the independent checkerboard calibration is retained rather than relying on the network’s own estimate.

Input frames are resized so the long edge is 518 px and the short edge is a multiple of the patch size, giving 518×294 for 16:9 content (777 tokens per frame) and 518×392 for 4:3 content (1036 tokens per frame). Portrait frames are rotated to landscape before inference and unrotated afterwards: an unrotated 9:16 frame would either lose 44% of its height to cropping or spend 43% of its tokens on padding, and the rotation reduces global-attention work by approximately a factor of three.

Numerical precision is chosen by compute capability. On Ampere and later the aggregator runs in `bfloat16`; below that it runs in `float16` and the heads are forced to

float32. This is not a conservative default but a measured necessity: the depth head’s confidence activation is $1 + \exp(\cdot)$, which overflows the float16 maximum of 65504 and returns NaN for every pixel. In bfloat16, which shares the exponent range of float32, depth differs from full precision by a median of 0.03%.

4.3. Window Sizing and Memory

Frames are processed in overlapping windows whose size is chosen to fill available video memory. Memory consumption is modelled analytically as a per-frame attention term plus a fixed dense-prediction term:

$$n = \left\lceil \frac{M_{\text{free}} - M_{\text{dpt}} - M_{\text{head}}}{\alpha T / 1000} \right\rceil \quad (9)$$

where $\alpha = 39.0$ MB per thousand tokens per frame, T is tokens per frame, M_{dpt} is 1230 MB per frame-megapixel over a 16-frame chunk, and $M_{\text{head}} = 1500$ MB of headroom. The result is clamped to $[12, 64]$. An out-of-memory condition shrinks the window by a factor of 0.75 and retries from the same start index, which is safe because pipeline state is mutated only after a successful forward pass.

4.4. Similarity Alignment Between Windows

Each window is solved independently and therefore carries its own arbitrary scale, rotation and origin. The composition of successive windows is the central problem, and the structure of the predictions makes it unusually tractable. Because consecutive windows share eight whole frames, and the network predicts a depth for every pixel of those frames in both windows, several hundred thousand exact correspondences are available with no feature detection or matching whatsoever.

Given corresponding point sets $\{\mathbf{x}_i\}$ and $\{\mathbf{y}_i\}$, the similarity transform minimising

$$E(s, \mathbf{R}, \mathbf{t}) = \sum_{i=1}^n \|s\mathbf{R}\mathbf{x}_i + \mathbf{t} - \mathbf{y}_i\|^2 \quad (10)$$

has the closed-form solution of Umeyama [5]. With centred points and the cross-covariance $\Sigma = \frac{1}{n} \sum_i (\mathbf{y}_i - \bar{\mathbf{y}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$ and its singular value decomposition $\Sigma = \mathbf{U}\mathbf{D}\mathbf{V}^\top$:

$$\mathbf{S} = \begin{cases} \mathbf{I} & \det(\mathbf{U}) \det(\mathbf{V}) \geq 0 \\ \text{diag}(1, 1, -1) & \text{otherwise} \end{cases} \quad (11)$$

$$\mathbf{R} = \mathbf{U}\mathbf{S}\mathbf{V}^\top, \quad s = \frac{\text{tr}(\mathbf{D}\mathbf{S})}{\sigma_x^2}, \quad \mathbf{t} = \bar{\mathbf{y}} - s\mathbf{R}\bar{\mathbf{x}} \quad (12)$$

The reflection correction is essential: without it the fit can return an improper rotation that mirrors the reconstruction.

Real correspondences contain gross outliers, principally at depth discontinuities and on moving objects, so the plain least-squares solution is not used directly. The implementation first draws 512 three-point hypotheses evaluated as a single batched decomposition against 20,000 sampled points, then refines the best hypothesis with ten iterations

of iteratively reweighted least squares under a Cauchy loss. Residuals are normalised by a local depth scale so that near and distant points contribute comparably:

$$r_i = \frac{\|s\mathbf{R}\mathbf{x}_i + \mathbf{t} - \mathbf{y}_i\|}{\rho_i}, \quad \sigma = 1.4826 \text{ median}(r) \quad (13)$$

$$w_i = \frac{1}{1 + (r_i/c\sigma)^2}, \quad c = 2 \quad (14)$$

The RANSAC seed is not redundant. Starting the reweighting from the plain least-squares answer converges to a compromise scale in the presence of one-sided outliers; a unit test reproduces exactly this, recovering 0.41 instead of the true 0.70 with 25% one-sided outliers.

Once $(s, \mathbf{R}, \mathbf{t})$ is known, the entire window is mapped into the global frame:

$$\mathbf{P}_g = s\mathbf{R}\mathbf{P} + \mathbf{t}, \quad \mathbf{C}_g = s\mathbf{R}\mathbf{C} + \mathbf{t} \quad (15)$$

$$\mathbf{R}_g = \mathbf{R}\mathbf{R}_{cw}, \quad d_g = s d \quad (16)$$

When fewer than 2000 confident shared pixels are available, the method degrades to rigid chaining from a single camera pose, with scale taken from the ratio of median depths. A full six-degree-of-freedom pose supplies exactly six constraints, so this fallback is well-posed, but it cannot observe scale robustly and its use is recorded as a warning in the report. Rotations are represented internally through the Rodrigues exponential map,

$$\mathbf{R} = \mathbf{I} + \sin \theta \mathbf{K} + (1 - \cos \theta) \mathbf{K}^2 \quad (17)$$

where $\mathbf{K}$ is the skew-symmetric matrix of the unit rotation axis, with a first-order approximation below 10^{-12} radians to avoid division by zero.

4.5. Depth Filtering and Point Generation

Not every predicted pixel is admissible. Three filters are applied.

First, an adaptive confidence gate. The threshold is the 50th percentile of the confidence distribution, but computed only over pixels strictly above the distribution’s floor. This subtlety was forced by a measured failure: on an overcast aerial clip 84% of all pixels sat at exactly the floor confidence, so a naive 50th percentile of the raw array *was* the floor, the gate admitted everything, and near-arbitrary sky depth survived into the cloud as a thin cone.

Second, a depth-discontinuity filter that removes points interpolated across object boundaries. The relative depth gradient is thresholded at 0.04 and dilated by a 3×3 maximum filter. The threshold is justified geometrically: a genuine surface viewed even at a grazing 80° changes depth by only about $\tan(80^\circ)/f_x \approx 1.4\%$ per pixel at this resolution, so anything steeper is a flying pixel.

Third, surface normals are estimated by central differences and cross product, and oriented toward the observing camera:

$$\mathbf{n} \leftarrow \begin{cases} -\mathbf{n} & \text{if } \mathbf{n} \cdot (\mathbf{C} - \mathbf{P}) < 0 \\ \mathbf{n} & \text{otherwise} \end{cases} \quad (18)$$

Consistent orientation is a precondition for Poisson reconstruction; with unoriented normals roughly half the surface can be reconstructed inside-out.

4.6. Voxel Fusion

Surviving points are accumulated into a sparse voxel grid whose resolution is tied to the imaging geometry rather than fixed in advance. The voxel edge is set to approximately the footprint of one pixel at the median scene depth:

$$v = \lambda \cdot \frac{\tilde{d}}{f_x} \quad (19)$$

with $\lambda = 1$ by default. Voxel indices are packed into a single 64-bit key using 21 bits per axis with an offset of 2^{20} , permitting accumulation entirely on the GPU through sorted unique reduction and indexed addition. Each voxel additionally tracks the number of *distinct* frames that observed it, and voxels seen by fewer than two frames are discarded, which removes single-view speculation. A statistical outlier filter then removes points whose mean distance to their twelve nearest neighbours exceeds two standard deviations of the global mean.

4.7. Surface Reconstruction

A surface is fitted to the oriented cloud by Screened Poisson reconstruction [6], at octree depth 10 with 1.5 samples per node and a point weight of 4.0. Poisson reconstruction produces a watertight surface by construction, which is a liability rather than an asset here: wherever the flight provided no data, it invents a smooth closed bubble. A distance trim is therefore applied, deleting every face whose vertices lie farther than

$$\tau = \max\left(3v, 2 \cdot \frac{1.1 \text{ extent}}{2^{\text{depth}}}\right) \quad (20)$$

from any real input point, followed by removal of small disconnected components. This makes the reconstruction's coverage gaps honest rather than plausible-looking.

4.8. Georeferencing

Positions are first converted from geodetic to a local east-north-up frame. A deliberately simple spherical approximation is used:

$$\Delta\phi = \frac{\Delta y}{R} \cdot \frac{180}{\pi}, \quad \Delta\lambda = \frac{\Delta x}{R \cos \phi_0} \cdot \frac{180}{\pi} \quad (21)$$

with $R = 6378137$ m. The forward and inverse conversions are exact algebraic inverses of one another, verified to close to 5×10^{-10} m. A rigorous ellipsoidal conversion would differ by roughly 0.3% in northing at the test latitude, about one metre over a 300 m scene; the design judgement taken is that one self-consistent approximation is safer than two mutually inconsistent accurate-looking ones, since the latter produces a metre-scale error with no symptom.

For export, coordinates are projected to Universal Transverse Mercator using the closed-form Transverse Mercator series of Snyder [13] with scale factor $k_0 = 0.9996$ on

the WGS-84 ellipsoid. With $e'^2 = e^2/(1 - e^2)$, $N = a/\sqrt{1 - e^2 \sin^2 \phi}$, $T = \tan^2 \phi$, $C = e'^2 \cos^2 \phi$ and $A = \cos \phi (\lambda - \lambda_0)$, the easting is

$$E = k_0 N \left(A + (1 - T + C) \frac{A^3}{6} + (5 - 18T + T^2 + 72C - 58e'^2) \frac{A^5}{120} + 500000 \right) \quad (22)$$

The whole trajectory is then anchored by a single robust similarity fit between the reconstructed camera centres and their corresponding satellite-navigation fixes, rather than a per-window fit. A global fit is used because scale and heading are weakly observed over the short baseline of one window. The fit is abandoned, and the model left in relative units, if fewer than three frames carry a fix within one second or if the fixes span less than five metres.

The accuracy consequence follows directly from least-squares theory: the fit cannot be better than the noise it is given, and the residual position error falls only as

$$\epsilon \propto \frac{\sigma}{\sqrt{N}} \quad (23)$$

in the number of anchors N . Section 7 quantifies this.

4.9. Why Inertial Fusion Was Removed

An earlier version of this system carried a fifteen-state error-state Kalman filter over position, velocity, attitude, accelerometer bias and gyroscope bias, with satellite-navigation updates, zero-velocity updates and Mahalanobis-gated vision fusion. It was removed. The reason was not a defect in the filter, whose own test suite passed throughout, but a measurement: every real capture recorded zero satellite-navigation fixes actually fused. Deprived of any absolute correction, the filter ran as pure inertial dead reckoning, in which uncorrected accelerometer bias double-integrates into a position error growing in a fixed direction and uncorrected gyroscope bias curves the heading. That is a guaranteed failure mode, not a tuning problem.

Feeding the vision pose back into the filter made matters worse, not better, degrading absolute trajectory error by a factor of 13 to 18, because the map against which the pose was resolved had itself been triangulated from the filter's own earlier poses and was therefore not independent information.

One inertial signal was retained. Gyroscope samples are integrated between frames into a relative rotation, used solely as a prior in windowed bundle adjustment in the legacy path, whose sparse front end tracks Shi–Tomasi corners [9] using Lucas–Kanade optical flow. Its justification is a sensitivity relation specific to plane-sweep stereo: the depth error induced by a relative attitude error $\delta\theta$ is

$$\Delta Z = \frac{Z^2 \delta\theta}{B} \quad (24)$$

At $Z = 85$ m over a $B = 15$ m baseline, a measured 1.99° of relative attitude error accounts for approximately 16 m of depth error, which is what the end-to-end cloud actually exhibited. Two independent checks confirmed the diagnosis

rather than assuming it. Driving the same stereo code with ground-truth poses gave a point-to-surface median of 0.29 m, so the stereo geometry itself was sound. Lengthening the baseline did not help, giving 20.0 m of error at a 4–20 m baseline and 19.5 m at 35–80 m, whereas a translation-driven error would have fallen as $1/B$.

4.10. Live and Batch Timing

Reconstruction may begin while capture is still in progress. This does not make inference faster, and the system does not claim that it does. The saving is structural. In batch operation, processing cannot begin until the recording is complete:

$$t_{\text{batch}} = t_{\text{capture}} + t_{\text{infer}} \quad (25)$$

whereas in live operation the two overlap:

$$t_{\text{live}} = \max(t_{\text{capture}}, t_{\text{infer}}) + t_{\text{window}} \quad (26)$$

The benefit is real whenever inference is the longer stage, which is usually the case, and is bounded by the shorter stage. It is never a reduction in GPU work.

5. HARDWARE IMPLEMENTATION

Component	Specification
GPU	NVIDIA RTX 5070 Ti, 16 GB
Earlier GPU	NVIDIA GTX 1650, 4 GB
OS	Windows 11 Pro 10.0.26200
Python	3.13.15
PyTorch	2.12.0.dev+cu128
OpenCV	5.0.0.93
Drone	Suparna, ArduPilot autopilot
Drone camera	Fisheye, 1280×720 @ 30 fps
Telemetry	MAVLink v2, TCP port 14550
Video	RTSP over TCP, port 10000
Handset	vivo I2206, Android 14
Handset camera	1080×1920 , CameraX 1.3.1
Model	VGGT-1B, approx. 5 GB checkpoint

Table 1: Hardware and platform inventory of the reference system.

5.1. Drone Subsystem

The aerial platform is a Suparna drone running ArduPilot [16], normally operated through a proprietary ground-control station. RTVIO reaches the aircraft directly using the same two channels that station uses: MAVLink v2 [15] over TCP to port 14550 for telemetry, and an RTSP video stream on port 10000. The video is carried over TCP rather than UDP because RTP over UDP loses packets, and therefore smears frames, on any lossy wireless link.

The MAVLink codec is implemented from first principles rather than through an existing library, covering twelve message types with their wire layouts and CRC extra bytes, using the CRC-16/MCRF4XX checksum whose standard check value $0x6F91$ is asserted in the test suite. The system presents itself as a ground station with system identifier 255, emits a heartbeat every second, and on connection requests elevated message rates.

Message	ID	Rate (Hz)
GLOBAL_POSITION_INT	33	10
ATTITUDE	30	10
GPS_RAW_INT	24	4
VFR_HUD	74	4
SYS_STATUS	1	4
BATTERY_STATUS	147	4

Table 2: Requested MAVLink telemetry rates. At a 5 m/s survey speed, 4 Hz leaves 1.25 m between position fixes and 10 Hz leaves 0.5 m, below the one-metre accuracy target.

A timing subtlety affects georeferencing accuracy. Video frames and position fixes are both timestamped by the workstation on arrival, so RTSP latency makes every frame late relative to its true position by a near-constant offset. At 5 m/s, 300 ms of latency corresponds to approximately 1.5 m of along-track error. The offset is therefore exposed as a configurable compensation, measured once per installation.



Figure 4: The Suparna aerial platform used in this work.

5.2. Handset Subsystem

The Android application captures JPEG frames through CameraX, inertial samples at 50–200 Hz, and satellite-navigation fixes from the platform location provider. The raw provider is used deliberately in preference to the fused provider, because fused location blends in WiFi and cellular positioning and produces plausible-looking fixes that may be wrong by metres to hundreds of metres.

One detail materially affects attitude quality. The wire format carries a single timestamp for a combined accelerometer and gyroscope sample, but Android delivers the two sensors independently. Rather than pairing each accelerometer sample with the most recent gyroscope reading, the gyroscope is linearly interpolated onto the accelerometer timestamp. The naive pairing leaves the gyroscope up to one sample period stale; at 100 Hz that is 10 ms, which during a $90^\circ/\text{s}$ pan injects a 0.9° attitude error into every sample.

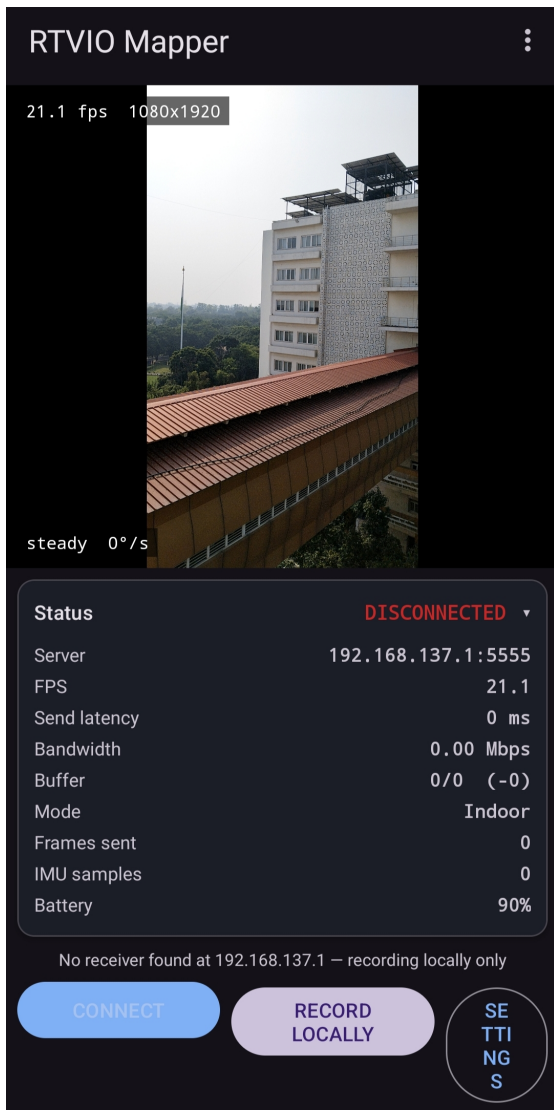


Figure 5: Android capture application: viewfinder with live frame rate, satellite-navigation state and stability overlay.

5.3. Lens Calibration Target

Calibration uses a printed planar checkerboard. Views are accepted only when the board occupies a genuinely new position, tracked on a 3×3 coverage grid; at least ten views are required and capture stops at twenty-five.

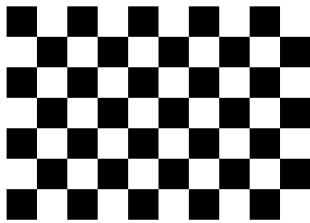


Figure 6: The A4 checkerboard calibration target. Only the count of inner corners matters; the physical square size does not affect the recovered lens model.

6. SOFTWARE IMPLEMENTATION

6.1. Wire Protocol

Handset and workstation communicate over a single big-endian TCP connection, dispatched on a one-byte header.

Header	Packet	Overhead
0xFF	Video frame	21 B + JPEG
0xFE	Inertial batch	32 B/sample
0xFD	Position fix	33 B
0xFC	Camera intrinsics	57 B + string
0xFA	Viewfinder preview	21 B + JPEG
0xFB	Status (JSON)	variable
0xC0	Command (JSON)	variable
0xAA	Handshake acknowledgement	6 B
0xE0–0xE2	Bulk session transfer	variable

Table 3: Packet types of the capture protocol.

Three clock domains must be reconciled: inertial samples are stamped in monotonic nanoseconds since device boot, while frames and position fixes carry epoch milliseconds. The offset is estimated with a minimum filter over observed receive lags rather than by pairing first packets, because transport delay is strictly positive and jittery, so the minimum over many observations is a better estimator than any single pairing.

6.2. Camera Intrinsics Discovery

Intrinsics are obtained automatically from the Camera2 API, preferring the factory-measured calibration where the hardware exposes it, falling back to computation from focal length and physical sensor size, and finally to a nominal guess. The recorded source string accompanies the values so that the provenance of any reconstruction is auditable.

Two real defects were found and corrected here, and are worth recording because both produce silently wrong geometry rather than an error. The reported calibration is expressed in active-array pixels, typically 4000×3000 , not in the dimensions of the transmitted buffer; and the transmitted frames are portrait while the sensor is landscape. The calibration is therefore mapped through the centred crop and scale, $k = \min(a_w/b_w, a_h/b_h)$, and then through the same rotation the encoder applies. A captured artefact from before the fix records $f_x = f_y = 3161.72$ with a principal point at the origin; after the fix the same device reports $f_x = f_y = 1487.87$ with $(c_x, c_y) = (539.0, 960.0)$.

6.3. Reconstruction Engine

Frame decoding runs on an eight-thread pool whose work is issued before the GPU forward pass, so decoding of the next window overlaps inference on the current one. Voxel merging runs on a background thread every six windows so the GPU never waits on host-side reduction. Window processing itself is synchronous, a deliberate choice to avoid two overlapping CUDA contexts.

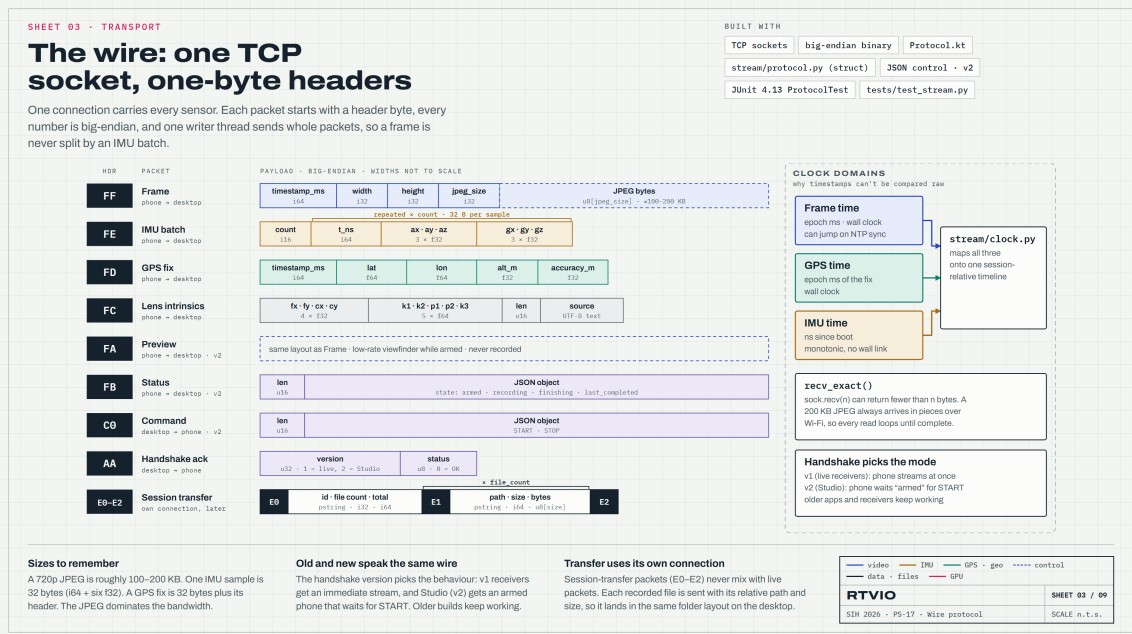


Figure 7: Wire protocol and the reconciliation of three independent clock domains: monotonic-since-boot inertial timestamps, and epoch-referenced frame and position timestamps.

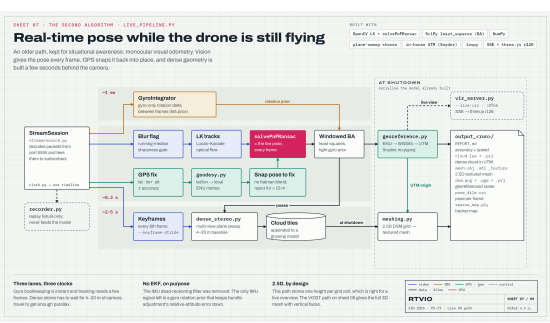


Figure 9: The legacy real-time path, retained for in-flight situational awareness: three lanes operating at approximately 1 ms, 0.3 s and 2-5 s of latency respectively.

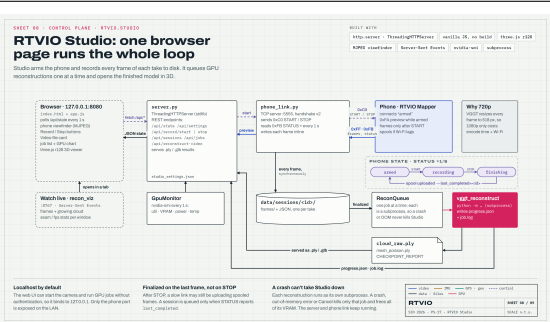


Figure 10: Studio control plane: capture sources, the single-occupancy GPU job queue, and the result viewer.

6.4. Control Plane

RTVIO Studio is served by Python's standard-library threading HTTP server with a vanilla JavaScript front end and a vendored three.js renderer, requiring no build step. It binds to the loopback interface by default because it can start a remote camera and launch GPU jobs while carrying no authentication.

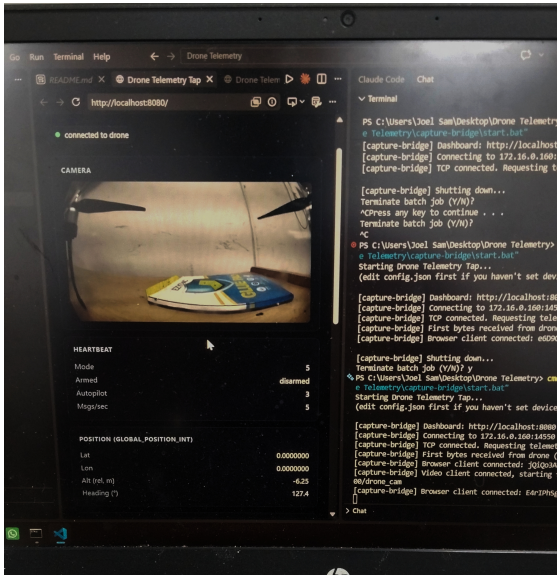


Figure 11: Studio Drone tab: live video, telemetry readout and the Indoor/Outdoor switch that determines georeferencing.

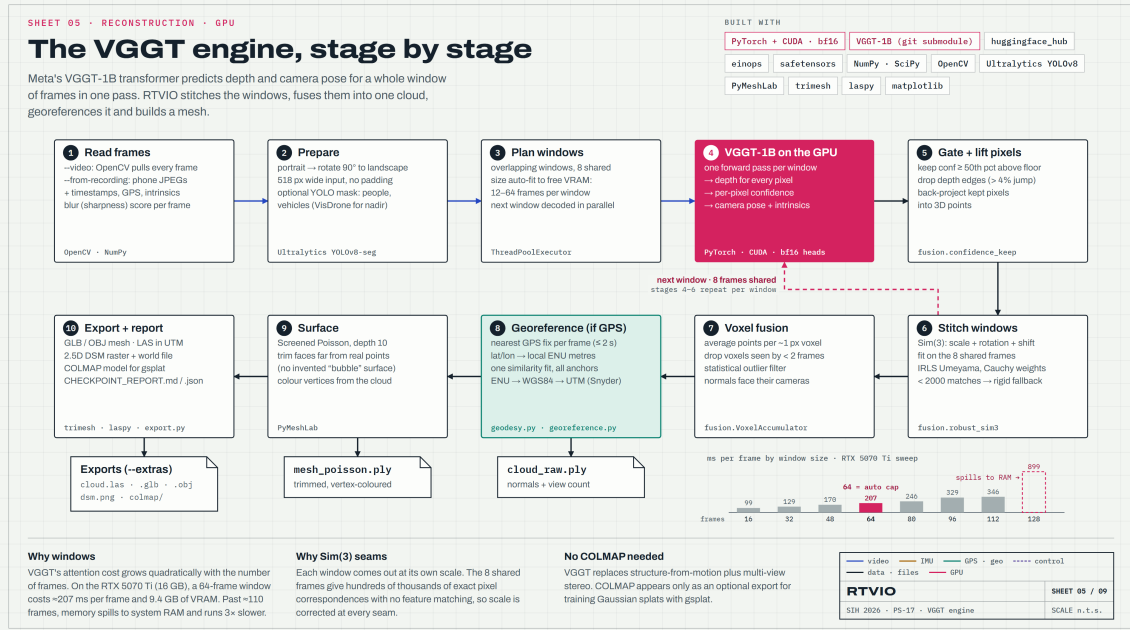


Figure 8: The ten-stage reconstruction engine, with the measured relationship between window size, per-frame inference cost and peak memory.

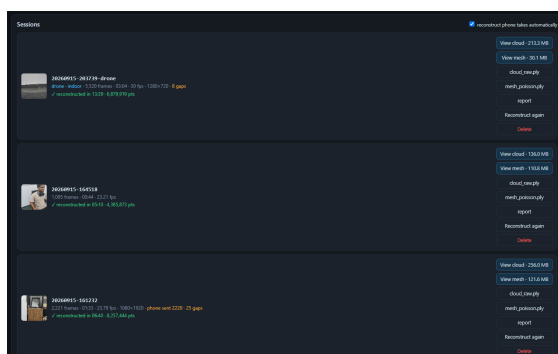


Figure 12: Session list with a reconstruction in progress and the GPU utilisation trace.

7. EXPERIMENTAL RESULTS

7.1. Reconstruction Throughput and Output

Seven complete reconstructions were performed on the reference workstation at default settings: 64-frame windows, 8 frames of overlap, no frame skipping, 50th-percentile confidence gating, a two-view minimum and Poisson depth 10.

Throughput is confined to a narrow band of 4.4 to 7.0 frames per second across a thirty-fold variation in sequence length, confirming that cost scales linearly in frames once window size is fixed. Re-running an identical session changed the point count by 0.08% and left seam scales unchanged, so the pipeline is effectively deterministic.

7.2. Window Size, Speed and Memory

Reconstructions execute one at a time, each in a separate subprocess, so that a crash or out-of-memory condition frees all video memory without affecting the server. Progress is published through an atomically rewritten JSON file, and GPU utilisation is sampled once per second.

6.5. Implementation Constraints

A notable characteristic of this implementation is how much standard machinery is absent. COLMAP, Open3D, GT-SAM, pyproj, rasterio and pymavlink are all unavailable as prebuilt wheels for Python 3.13 on Windows. Consequently the Universal Transverse Mercator projection, the similarity estimator, the statistical outlier filter, the plane-sweep stereo matcher, the bundle adjuster and the MAVLink codec are all implemented directly against NumPy, SciPy and OpenCV.

Frames/window	ms/frame	Peak VRAM (GB)
16	99	7.5
32	129	8.1
48	170	8.8
64	207	9.4
80	246	10.0
96	329	10.7
112	346	11.3
128	899	12.3

Table 5: Window-size sweep at 1036 tokens per frame. The card is fully utilised at every size; per-frame cost rises because global attention is quadratic in frame count, while cross-window consistency improves. Past roughly 110 frames the driver begins spilling to system memory, which raises no error and simply runs three times slower.

The default of 64 frames is the compromise this table motivates: large enough that consecutive windows share sub-

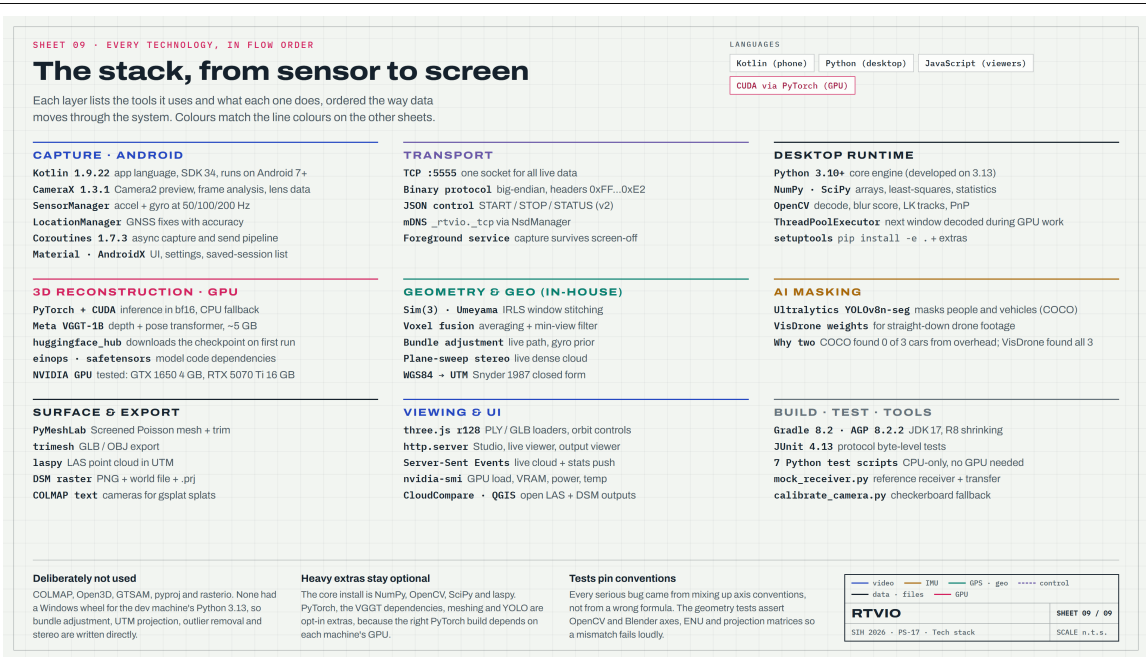


Figure 13: The complete technology stack from sensor to screen, including the components deliberately not used.

Run	Frames	Capture (s)	f/s	VRAM (MB)	Wall (s)	Points	Faces	Seam median
Quickstart, 4K clip	201	8.0	6.4	7981	80.6	603,662	1,326,556	0.988
Video job, 4K clip	201	8.0	6.4	7981	53.0	603,498	1,328,261	0.988
Video job, portrait	2,093	34.9	6.9	8342	343.1	6,579,423	2,066,247	0.564
Video job, portrait	6,891	114.9	6.8	7983	1106.1	16,939,287	4,254,186	0.522
Handset via Studio	2,221	93.3	6.9	7983	398.1	8,257,444	4,567,172	0.996
Handset, offline	1,095	44.2	4.4	9512	305.0	4,382,200	4,177,734	0.481
Drone, RTSP	5,520	184.0	7.0	7983	817.6	6,879,919	1,125,829	0.656

Table 4: Seven complete reconstructions on an NVIDIA RTX 5070 Ti. Throughput spans 4.4–7.0 frames per second, corresponding to $0.10\text{--}0.23\times$ real time. A seam median near 1.0 indicates the network maintained consistent scale across windows.

stantial context, small enough to stay well clear of the spill threshold.

7.3. Fusion Selectivity

The drone reconstruction illustrates the degree of filtering applied between raw prediction and final geometry:

Stage	Count
Confident pixels	119,986,846
Occupied voxels	11,629,189
Seen by ≥ 2 frames	7,145,097
After outlier filter	6,879,919
Poisson faces	1,419,391
After distance trim	1,125,829

Table 6: The fusion funnel for the 184-second drone take. Approximately 94% of confident pixels are absorbed by voxel merging, and 21% of Poisson faces are discarded as invented surface.

7.4. Georeferencing Accuracy

The accuracy of the satellite-navigation anchoring was characterised by Monte-Carlo simulation over 200 trials per configuration, using a synthetic flight along a 60° arc at 30 m

radius and 80 m altitude. A near-straight path is used deliberately, because the similarity fit is worse conditioned on a near-linear trajectory than on a well-spread point cloud, and a real survey path is close to a line. Vertical noise is set to twice horizontal, as is typical of consumer receivers.

Anchors	σ_h (m)	Mean (m)	Max (m)	≤ 1 m
4	0.0	0.000	0.000	yes
4	1.0	1.757	3.812	no
4	3.0	5.065	11.179	no
4	5.0	8.270	18.544	no
10	0.0	0.000	0.000	yes
10	1.0	1.086	2.030	no
10	3.0	3.147	6.117	no
10	5.0	5.106	10.241	no

Table 7: Monte-Carlo georeferencing error against the one-metre target, 200 trials per row. With noiseless anchors the estimator is exact to 1.6×10^{-14} m, confirming the implementation; with realistic consumer-receiver noise the target is missed at every level tested.

This is the most important negative result in this work, and it is a property of the measurement rather than of the estimator. With perfect anchors the fit recovers scale, rotation and

translation to 10^{-8} and places points to within 1.6×10^{-14} m. With one metre of horizontal noise and the four anchors a short window provides, the mean position error is 1.757 m, already above the target before any reconstruction error, timing error or lens error is considered. Increasing anchor count from four to ten reduces the error from 1.757 m to 1.086 m, consistent with the expected $\sigma/\sqrt{N}$ behaviour, but does not reach the target. A least-squares fit cannot be more accurate than the data it is given; the only effective remedy is a better position source, which is why RTK or PPK correction, listed as an optional input in the problem statement, is the principal recommendation arising from this analysis.

7.5. Hardware-Verified Capture

Route	Result	Rate
Handset streaming	1,137 frames, 52.9 s	21.5 f/s
Studio recording	2,221 frames, 93.3 s	23.8 f/s
Offline recording	2,863 frames, 118.9 s	24.1 f/s
Live reconstruction	579 frames, 101 s	5.7 f/s
Drone recording	5,520 frames, 184.0 s	30.0 f/s

Table 8: Capture routes verified on physical hardware. The drone take dropped zero frames across 5,520 frames and 6,828 telemetry samples.

The live reconstruction figure quantifies a real and initially underestimated cost. Because window processing blocks the receiving thread for eight to fifteen seconds at a time, the handset’s bounded transmit queue discards most frames captured during that interval, delivering 5.7 frames per second against the roughly 25 obtained by offline recording. The reconstruction completes correctly and is georeferenced, but from a visibly sparser set of views. Offline recording is therefore the recommended route whenever density matters more than turnaround.

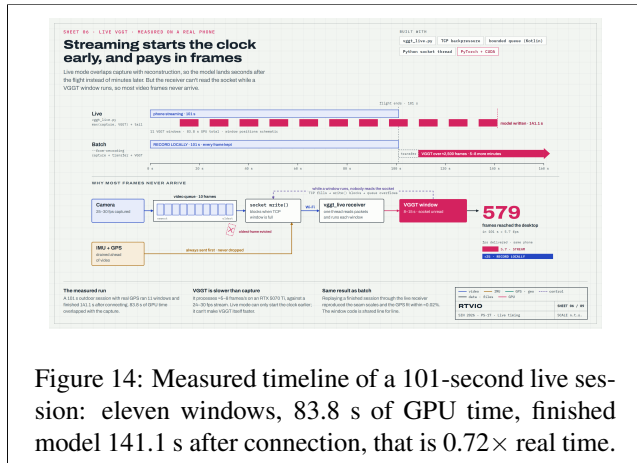


Figure 14: Measured timeline of a 101-second live session: eleven windows, 83.8 s of GPU time, finished model 141.1 s after connection, that is $0.72 \times$ real time.

7.6. Recovered Trajectories

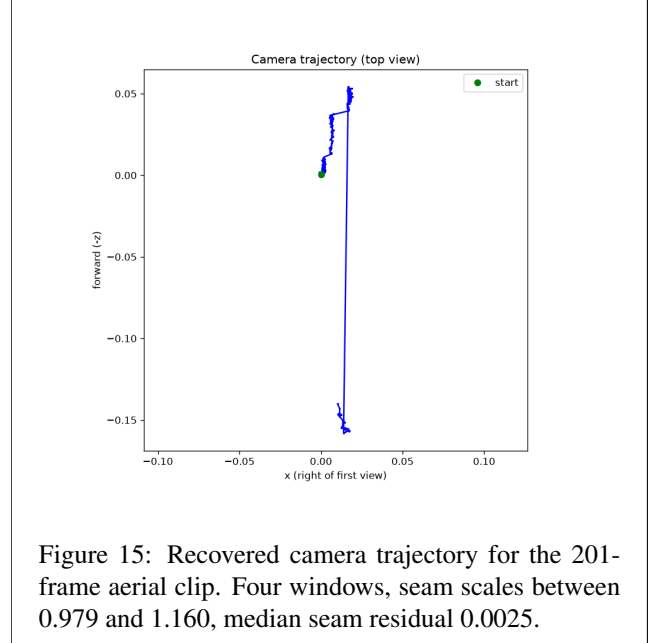


Figure 15: Recovered camera trajectory for the 201-frame aerial clip. Four windows, seam scales between 0.979 and 1.160, median seam residual 0.0025.

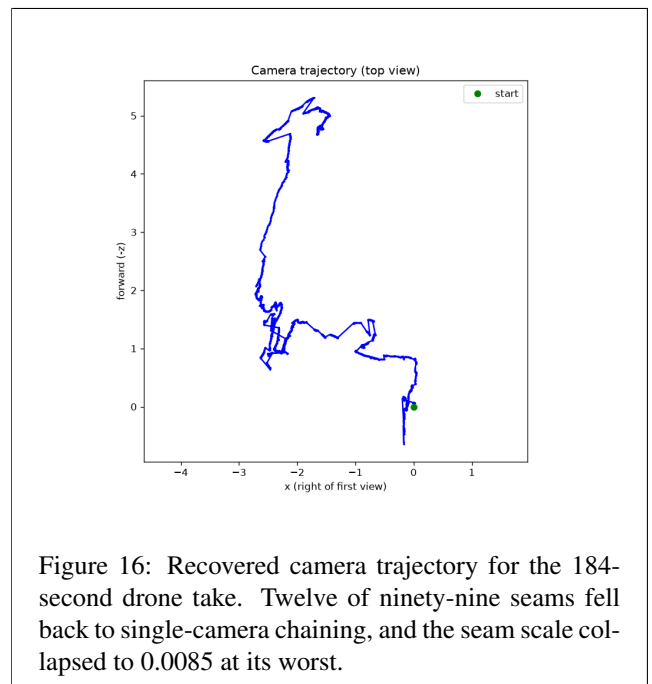


Figure 16: Recovered camera trajectory for the 184-second drone take. Twelve of ninety-nine seams fell back to single-camera chaining, and the seam scale collapsed to 0.0085 at its worst.



Figure 17: Dense coloured point cloud of the drone take, rendered in the Studio viewer.



Figure 18: Screened Poisson mesh of the same capture, shaded.

7.7. Verification

Correctness is maintained by 143 assertions across nine test suites, all CPU-only and executable without a GPU or the model checkpoint, together with 19 unit tests on the Android side. One suite exists solely to pin coordinate conventions, because every serious defect encountered in this project was a convention mismatch rather than an incorrect formula: a pose expressed in one camera axis convention consumed by code assuming another. One such mismatch produced three simultaneous silent failures, in which bundle adjustment became a no-op, the recovered attitude was 180° inverted, and the accuracy metric compared two unrelated coordinate frames. None of these raised an exception.

Lens calibration was validated against checkerboards rendered through a known 135° fisheye, recovering focal length within 1%, principal point within 3 pixels and field of view within 1° , and straightening an edge bowed by more than 15 pixels to under 0.2 pixels.

8. DISCUSSION AND LIMITATIONS

This section states the system's unresolved gaps explicitly, since several bear directly on the problem statement's acceptance criteria.

8.1. No Reconstruction Reported Here Is Georeferenced

Every reconstruction in Table 5 was produced in relative units. The control plane currently disables georeferencing by default, which suppresses automatic detection of position data in a recorded session. The georeferencing pathway is implemented and unit-tested, and the Monte-Carlo analysis of Section 7.4 characterises its error, but no end-to-end field measurement of metric accuracy has been performed. No claim of demonstrated one-metre accuracy is made.

8.2. Satellite-Navigation Accuracy

As quantified above, consumer-grade positioning noise alone places the anchoring error above one metre. This is a measurement-limited rather than an algorithmic gap, and RTK or PPK correction is the identified remedy.

8.3. Throughput Against the Budget

At 6.9 frames per second, a ten-minute video at 30 fps contains 18,000 frames and requires approximately 43 minutes of inference alone, against a fifteen-minute budget. Processing every third frame brings this to about 14.5 minutes, which meets the budget only before fusion and meshing are counted, and at the cost of reduced overlap. Speed remains the least satisfied of the problem statement's criteria. It is worth noting the trajectory: on the earlier 4 GB GPU the same workload projected to approximately 142 minutes, some 9.5 times over budget, so most of the gap has been closed by window sizing and hardware rather than by algorithmic change.

8.4. The Real Lens Has Never Been Calibrated

The calibration subsystem is complete and validated on synthetic fisheye renders, but it has never been run against the physical drone camera: no calibration profile exists on the reference system. The 184-second drone reconstruction consequently exhibits bowed walls and opened right angles, and its severe seam behaviour, with a worst-case scale of 0.0085 and twelve fallback seams, is at least partly attributable to feeding a strongly distorted fisheye image to a model that assumes a pinhole camera.

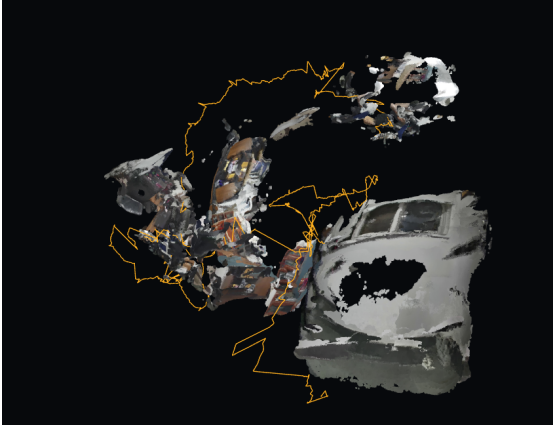


Figure 19: Consequence of an uncalibrated fisheye lens: wall surfaces bow and right angles open out.

8.5. Untested Outdoor Operation

The drone has been verified indoors only, carried by hand rather than flown, because pre-arm checks did not pass indoors. No outdoor flight with a valid position fix has been recorded, so the entire georeferenced drone pathway is untested in the field. The video latency compensation has likewise never been measured.

8.6. Single-Pass Geometry Is Incomplete by Construction

Surfaces never photographed cannot be reconstructed. A single overhead or oblique pass over a building sees its walls at a grazing angle if at all, so wall geometry is typically thin, sparse or absent, sometimes read as a floating roof. This is a property of any multi-view method, not a defect of the meshing stage, and the distance trim is designed to leave the gap visible rather than fill it with a plausible invention. The remedy is an orbital capture pass, not a post-processing heuristic.

8.7. Scale Drift

Where dense correspondence between windows is unavailable, the fallback cannot robustly observe scale, and any per-window scale error accumulates without correction. This has the same error shape as uncorrected inertial dead reckoning, differing only in that it affects scale rather than position.

9. FUTURE SCOPE

Survey-grade positioning. The single change with the largest expected effect on accuracy is RTK or PPK correction. The analysis of Section 7.4 shows the estimator is exact given exact anchors, so centimetre-grade positioning would move the error budget from the measurement to the reconstruction.

Loop closure. The present method aligns consecutive windows only. Detecting revisited geometry and imposing a global consistency constraint would bound the accumulated scale drift.

Tightly coupled inertial estimation. A genuine visual-inertial estimator with pre-integration factors and joint sliding-window optimisation, in the manner of VINS-Mono [10], would supply the metric scale that monocular vision inherently lacks.

Radiance-field rendering. The pipeline already exports a COLMAP model specifically so that 3D Gaussian Splatting [12] can be trained on the reconstruction, offering photorealistic rendering alongside the geometric model.

Distributed inference. Because windows are independent until the alignment step, inference parallelises naturally across multiple GPUs, with alignment performed afterwards. This is the most direct route to the processing budget.

Propeller and dynamic-object masking. Propeller blades intrude into the upper corners of every drone frame and warrant a fixed image mask. For dynamic objects, a general-purpose detector [14] proved unusable from directly overhead, detecting none of three clearly visible parked vehicles in nadir imagery, whereas an aerial-specific model detected them with confidences from 0.53 to 0.86.

10. CONCLUSION

This paper has presented RTVIO, an end-to-end system that converts a single-pass UAV video into a dense, textured three-dimensional model by replacing the conventional Structure-from-Motion and Multi-View Stereo pipeline with a single feed-forward geometry transformer. The principal technical contribution is the exploitation of dense per-pixel correspondence between overlapping inference windows, which permits a robust similarity transform to be fitted from hundreds of thousands of exact correspondences without any feature matching, correcting scale at every seam.

The system has been demonstrated end to end on real hardware across seven complete reconstructions, at 4.4 to 7.0 frames per second on a consumer GPU, producing clouds of up to 16.9 million points, and integrating capture from both a MAVLink drone and an Android handset behind a single browser-based control plane.

Two gaps remain, and both have been quantified rather than deferred. Consumer-grade satellite-navigation noise alone yields a mean anchoring error of 1.757 m at the window anchor count the pipeline actually uses, placing the one-metre accuracy target out of reach without corrected positioning. Inference throughput meets the fifteen-minute budget for a ten-minute video only with frame subsampling. Reporting these limits precisely, alongside the measurements that establish them, is intended to make the result reproducible and the remaining work well defined.

References

- [1] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, *VGGT: Visual Geometry Grounded Transformer*, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025. Best Paper Award.
- [2] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, *DUS3R: Geometric 3D Vision Made Easy*, Pro-

- ceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.
- [3] V. Leroy, Y. Cabon, and J. Revaud, *Grounding Image Matching in 3D with MAST3R*, European Conference on Computer Vision (ECCV), 2024.
 - [4] J. L. Schönberger and J.-M. Frahm, *Structure-from-Motion Revisited*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4104–4113, 2016.
 - [5] S. Umeyama, *Least-Squares Estimation of Transformation Parameters Between Two Point Patterns*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 13, no. 4, pp. 376–380, 1991.
 - [6] M. Kazhdan and H. Hoppe, *Screened Poisson Surface Reconstruction*, ACM Transactions on Graphics, vol. 32, no. 3, pp. 1–13, 2013.
 - [7] J. Kannala and S. S. Brandt, *A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 28, no. 8, pp. 1335–1340, 2006.
 - [8] Z. Zhang, *A Flexible New Technique for Camera Calibration*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 11, pp. 1330–1334, 2000.
 - [9] J. Shi and C. Tomasi, *Good Features to Track*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 593–600, 1994.
 - [10] T. Qin, P. Li, and S. Shen, *VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator*, IEEE Transactions on Robotics, vol. 34, no. 4, pp. 1004–1020, 2018.
 - [11] A. I. Mourikis and S. I. Roumeliotis, *A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation*, IEEE International Conference on Robotics and Automation (ICRA), pp. 3565–3572, 2007.
 - [12] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, *3D Gaussian Splatting for Real-Time Radiance Field Rendering*, ACM Transactions on Graphics, vol. 42, no. 4, 2023.
 - [13] J. P. Snyder, *Map Projections: A Working Manual*, U.S. Geological Survey Professional Paper 1395, United States Government Printing Office, Washington, D.C., 1987.
 - [14] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*, version 8.0.0, 2023. Available: <https://github.com/ultralytics/ultralytics>
 - [15] L. Meier et al., *MAVLink Micro Air Vehicle Communication Protocol*, 2009–2026. Available: <https://mavlink.io/>
 - [16] ArduPilot Development Team, *ArduPilot Autopilot Suite Documentation*. Available: <https://ardupilot.org/>
 - [17] Smart India Hackathon 2026, Problem Statement PS-17, *Single-Pass Drone Video to Accurate 3D Model Generation*, Government of India, 2026.